API Data Flow Reference

System overview

PetPals uses a two-token authentication system. Authentication credentials are centralised in a single USERS table shared across all six role-specific tables (Admin, Staff, Veterinarian, Adopter, Volunteer, Donor).

Token	Storage	Expiry	Purpose
Access token	Zustand memory (frontend)	15 minutes	Sent in Authorization header on every protected API request.
Refresh token	httpOnly cookie (browser)	7 days	Used only to issue new access tokens - never sent in API headers

Architecture pattern followed by every backend endpoint:

```
Route → Middleware (protected routes only) → Controller → Service → Database
```

Frontend auth pattern on every page load:

```
App.tsx mounts → refreshToken() called → httpOnly cookie sent → new access
token returned → Zustand populated
```

> *Sprint 9 - email and account security: the Users table now carries emailVerified, and login and refresh-token return it for every role. Nine new /auth endpoints (email verification, password reset, password change, account reactivation, email change) and two new account statuses (Closed, Declined) are covered in 1.8 to 1.16. The emails themselves are covered in 16 - Email & Account Security.*

1. Backend Auth API

Base path: /api/v1/auth

Thirteen endpoints: the original four covering registration, login, logout, and token refresh (1.1 to 1.4), plus nine added in Sprint 9 for email verification, password reset, password change, account reactivation and email change (1.10 to 1.14). Register and login are public. Logout requires a valid Bearer access token. Refresh-token requires only the httpOnly cookie. Of the Sprint 9 endpoints, PATCH /email-verification, POST and PATCH /password-reset, POST and PATCH /reactivation and PATCH /email-change are public (the emailed link is the proof); POST /email-verification, PATCH /password and POST /email-change need a Bearer access token. The statuses, token mechanism, verification gate and security decisions behind them are in 1.8, 1.9, 1.15 and 1.16.

1.1 POST /api/v1/auth/register

Creates a new user account. Writes two rows atomically - one to USERS (credentials) and one to the role-specific table (name + userID). The Prisma $transaction ensures both succeed or both roll back.

Request summary

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	name, email, password, role
Valid roles	admin, adopter, staff, vet, volunteer, donor
Password rules	8+ characters with an uppercase letter, a lowercase letter, a number and a special character, enforced server-side (see 1.16)
Success	201 - { userID, userEmail, role }

Data flow

File / Layer	What happens here
auth.routes.js	Receives POST request, no middleware – public route. Maps POST /register → register controller
auth.controller.js	Validates all 4 fields are present and role is one of the 6 valid values. Destructures { name, email, password, role } from req.body. Returns VALIDATION_ERROR (422) if any field is missing. Returns VALIDATION_ERROR (422) if role is not valid. Calls authService.register() in try/catch. On success: successResponse 201 with created user. On failure: next(err) → errorHandler
auth.service.js	Duplicate email check, bcrypt hash, atomic double-insert via Prisma transaction. findUnique on USERS by email - throws CONFLICT (409) if exists. bcrypt.hash(password, 10). Opens $transaction: creates USERS row, then role-specific row using returned userID. If either insert fails, entire transaction rolls back - no orphaned records. Strips userPassword from returned object
USERS table	Stores auth credentials. Columns written: userEmail, userPassword (hashed), role. The table also carries emailVerified (added in Sprint 9; existing accounts were backfilled as true) - see 1.10
Role table	Stores name and userID for the specific role. Columns written: userID (FK → USERS), [role]Name (e.g. adopterName)

> *accountStatus at registration: accountStatus is set explicitly for four of the six roles - Admin, Staff, Veterinarian, and Volunteer - which start as "Pending" and require approval (via the PATCH .../status endpoints) before they can operate. The very first Admin registered in the system auto-activates as "Active" immediately, since there is no existing admin to approve them. Adopter and Donor registrations start "Active" by their own table defaults. Staff, Vet, and Volunteer choose a shelter at sign-up (shelterID is required), and the shelter must be Open (422 otherwise). A vet can register only at a shelter that has a manager (422 "This shelter isn't onboarding veterinarians yet - it has no manager assigned"). The first Staff sign-up at a shelter with no manager (and no pending Manager sign-up) registers as that shelter's Manager and is approved by an Admin; other staff and vets are approved by that shelter's Manager, and volunteers by any staff member at their shelter. Staff, Veterinarians and Volunteers onboard BEFORE approval: after registering they complete an onboarding wizard (personal details, address, government ID) - see 1.6. Staff and Veterinarians are signed straight in by the worker-portal sign-up page; Volunteers are not workers and use the public /login page. Admin accounts wait for approval before they can sign in. Adopter and Donor accounts are already Active and have a skippable onboarding wizard (see 1.7).*
>
> *Email verification at registration: registering sends the first email-verification link (see 1.10). Accounts that existed before Sprint 9 were backfilled as verified.*

Error cases

Scenario	Error Code	Response Message
Missing any of the 4 fields	VALIDATION_ERROR	name, email, password, and role are required
Role not in the allowed list	VALIDATION_ERROR	role must be one of: admin, adopter, staff, vet, volunteer, donor
Shelter is not Open (Staff or Vet sign-up)	VALIDATION_ERROR	Shelter is not Open
Vet signs up at a shelter with no manager	VALIDATION_ERROR	This shelter isn't onboarding veterinarians yet - it has no manager assigned
Email already registered	CONFLICT	A user with this email is already registered
Password too weak	VALIDATION_ERROR	error.details.unmet lists the missing rules (see 1.16)
DB insert fails	INTERNAL_SERVER_ERROR	Internal server error

1.2 POST /api/v1/auth/login

Authenticates an existing user. Verifies credentials against USERS, issues a 15m access token in the response body and a 7d refresh token in an httpOnly cookie. Also fetches the user's name from the role-specific table for UI display.

Request summary

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	email, password
Success	200 - { token (15m), user: { userID, userEmail, role, name, avatarSeed, emailVerified (every role), onboardingComplete, onboardingStep (Adopter/Staff/Veterinarian/Volunteer/Donor), accountStatus (Staff/Veterinarian/Volunteer) } } + refreshToken httpOnly cookie (7d)

Data flow

File / Layer	What happens here
auth.routes.js	Receives POST request, no middleware - public route. Maps POST /login → login controller
auth.controller.js	Validates fields, calls service, signs both tokens, sets cookie. Returns VALIDATION_ERROR (422) if email or password missing. Calls authService.login() in try/catch. Signs access token payload: { userID, role } for all roles, 15m expiry, JWT_SECRET. Signs refresh token: { userID }, 7d expiry, JWT_SECRET. Hashes refresh token with bcrypt and calls authService.storeRefreshToken(userID, hashedRT). Sets refreshToken as httpOnly cookie (sameSite: strict, secure in production). Returns 200 with { token: accessToken, user }
auth.service.js (login)	Verifies credentials, fetches name from role table, returns safe user object. findUnique on USERS by email (selects userID, userEmail, role, userPassword and emailVerified only). Throws NOT_FOUND (404) if no record. bcrypt.compare(password, user.userPassword) - throws UNAUTHORIZED (401) if no match. Queries role-specific table for the user's name using ROLE_CONFIG. Checks the account status: Deactivated, Banned, Closed and Declined accounts are refused (a Closed or Declined account carries error.details.reason "account_closed" or "account_declined" - see 1.8), and Pending accounts are refused unless the role is in PENDING_LOGIN_ROLES (Staff, Veterinarian, Volunteer). Strips userPassword before returning { ...safeUser, name }
auth.service.js (storeRefreshToken)	Stores hashed refresh token in USERS table. prisma.users.update({ where: { userID }, data: { refreshToken: hashedRT } })
USERS table	Queried for credentials, updated with refresh token hash. Columns read: userEmail, userPassword, emailVerified. Columns written: refreshToken (hashed)
Role table	Queried for the user's display name. e.g. Adopter.adopterName, Staff.staffName etc. avatarSeed is also selected here for every role (not Staff-specific) and included in the returned user object. For Adopter, Staff, Veterinarian, Volunteer and Donor the onboarding progress (onboardingComplete, onboardingStep) is selected too, and accountStatus for Staff, Veterinarian and Volunteer.

> *Design note — shelterID is not in the token: the payload is { userID, role } for every role. Staff-scoped endpoints re-fetch the staff member's current shelterID from STAFF on each request, so a shelter reassignment takes effect immediately.*

> *lastLoginAt lives on USERS and is updated on every successful login, for all roles.*

Error cases

Scenario	Error Code	Response Message

Missing email or password	VALIDATION_ERROR	email and password are required
Email not found in USERS	NOT_FOUND	No account found with this email
Password does not match hash	UNAUTHORIZED	Incorrect password
Account status is Pending (Admin)	UNAUTHORIZED	This account is pending approval and can't log in yet.
Account status is Deactivated or Banned (every role)	UNAUTHORIZED	This account is no longer active
Account status is Closed (closed by its owner)	UNAUTHORIZED	Own message; error.details.reason = "account_closed" - the login page offers a reactivation link (see 1.13)
Account status is Declined (turned down at approval)	UNAUTHORIZED	Own message; error.details.reason = "account_declined" - final, no reactivation (see 1.8)

> *Pending Staff, Pending Veterinarian and Pending Volunteer accounts can log in: login succeeds so they can onboard before approval, but authenticate admits them only on their own onboarding routes (see 1.5). The roles whose Pending accounts can log in are defined once as PENDING_LOGIN_ROLES = { Staff, Veterinarian, Volunteer } in server/src/services/auth/pendingRoles.js, shared by login() in auth.service.js and the authenticate middleware. Pending Admin accounts cannot log in (401), Deactivated and Banned accounts are refused for every role, and Closed and Declined accounts are refused after a correct password (see 1.8). Every login refusal is a 401, never a 403: the frontend's axios interceptor redirects every 403 to /forbidden, which would hide the message on the login page. The success response carries emailVerified for every role, onboardingComplete and onboardingStep for Adopter, Staff, Veterinarian, Volunteer and Donor, and accountStatus for Staff, Veterinarian and Volunteer. A Donor, like an Adopter, carries no accountStatus: Donors are Active from sign-up.*

1.3 POST /api/v1/auth/logout

Ends the session by nullifying the stored refresh token in the DB and clearing the httpOnly cookie. The access token is not denylisted - with a 15-minute expiry the risk window is acceptable. The refresh token nullification is the source of truth for session invalidation.

Request summary

Field	Value
Method	POST
Auth required	Yes — Bearer access token (a Pending Staff member, Veterinarian or Volunteer may log out too)
Body fields	None
Success	200 - { message: "Logged out successfully" }

Data flow

File / Layer	What happens here
auth.routes.js	Registers authenticate middleware before controller. Maps POST /logout to: authenticate.allowPending → logout controller
authenticate.js	Validates the Bearer access token. Extracts token from Authorization header via optional chaining. Returns UNAUTHORIZED if missing. jwt.verify(token, JWT_SECRET) - throws if invalid or expired. Attaches { userID, role } to req.user, checks the account status live (see 1.5), and calls next()
auth.controller.js	Calls service with userID and clears the cookie. Uses req.user.userID (set by authenticate middleware). Calls authService.logout(userID) in try/catch. res.clearCookie('refreshToken') - removes httpOnly cookie from browser. Returns 200 with 'Logged out successfully'
auth.service.js	Nullifies stored refresh token in DB. prisma.users.update({ where: { userID }, data: { refreshToken: null } }). Any future refresh attempt will fail bcrypt comparison
USERS table	refreshToken column set to null. Columns written: refreshToken = null

Error cases

Scenario	Error Code	Response Message
No Authorization header	UNAUTHORIZED	Authentication failed - no token provided
Access token invalid or expired	UNAUTHORIZED	Authentication failed - invalid token

1.4 POST /api/v1/auth/refresh-token

Issues a new access token and rotates the refresh token. Does not require an access token - authentication is via the httpOnly cookie only. Called by the frontend on every app load to restore session from the cookie.

Request summary

Field	Value
Method	POST
Auth required	No - cookie only (no Bearer token needed)
Cookie required	refreshToken httpOnly cookie
Body fields	None
Success	200 - { token (new access token, 15m), user: { userID, role, name, avatarSeed, emailVerified (every role), onboardingComplete, onboardingStep (Adopter/Staff/Veterinarian/Volunteer/Donor), accountStatus (Staff/Veterinarian/Volunteer) } } + new refreshToken cookie (7d)

Data flow

File / Layer	What happens here
auth.routes.js	No middleware - cookie-only authentication handled in controller. Maps POST /refresh-token → refreshToken controller directly
auth.controller.js	Reads cookie, decodes userID from it, signs new tokens, calls service. Reads rawOldRT from req.cookies.refreshToken - returns UNAUTHORIZED if missing. jwt.decode(rawOldRT) to extract userID (no signature verification here). Returns UNAUTHORIZED if decoded.userID is absent. Signs new refresh token: { userID }, 7d expiry. Calls authService.refreshToken(userID, rawOldRT, newRefreshToken) in try/catch. On success: signs new access token payload { userID, role }, 15m expiry. Sets new refreshToken as httpOnly cookie. Returns 200 with new access token and { userID, role, name, avatarSeed, emailVerified, onboardingComplete, onboardingStep, accountStatus } (emailVerified for every role; the onboarding fields for Adopter, Staff, Veterinarian, Volunteer and Donor; accountStatus for Staff, Veterinarian and Volunteer). Design note: this response object is built from an explicit field whitelist in the controller ({ userID, role, name, avatarSeed, emailVerified, onboardingComplete, onboardingStep, accountStatus }). Any field added to the service's return value must also be added to this whitelist explicitly, or it is dropped without error.
auth.service.js	Verifies old refresh token against DB hash, stores new hash, fetches name and avatarSeed. findUnique on USERS by userID (selects userID, role, refreshToken). Throws NOT_FOUND if user missing. Throws UNAUTHORIZED if refreshToken in DB is null (already logged out). bcrypt.compare(rawOldRT, user.refreshToken) - throws UNAUTHORIZED if no match. bcrypt.hash(rawNewRT, 10) and updates USERS.refreshToken with new hash. Queries role-specific table for the user's name and avatarSeed using ROLE_CONFIG. Returns { userID, role, name, avatarSeed } (refreshToken stripped).
USERS table	refreshToken read for verification, updated with new hash. Columns read: refreshToken (hash for bcrypt comparison), role. Columns written: refreshToken (new hash)

Error cases

Scenario	Error Code	Response Message
No refreshToken cookie	UNAUTHORIZED	No refresh token provided
Cookie cannot be decoded	UNAUTHORIZED	Invalid refresh token
User not found in DB	NOT_FOUND	User not found

refreshToken null in DB (logged out)	UNAUTHORIZED	Session expired, please log in again
Cookie does not match DB hash	UNAUTHORIZED	Invalid refresh token

Design note - rotation

Each refresh token can only be used once. After a successful refresh the old hash is replaced in the DB. If an attacker steals a refresh token and tries to use it after the legitimate user has already rotated it, bcrypt.compare will fail and the request is rejected.

1.5 Live Account-Status Enforcement

Every protected request verifies the Bearer token and then checks the account's status live, so an access token never outlives the account's standing. middleware/authenticate.js calls the auth.service.js export getAccountStatus(userID, role) on every protected request (one indexed lookup on the role table's primary key), not only at login and refresh. If the account's current status is Deactivated, Banned, Pending, or the sentinel "DELETED" (returned when the role's row no longer exists at all - e.g. after a completed self-delete), the request is rejected with 401 UNAUTHORIZED, message "This account is no longer active", even if the JWT signature and expiry are otherwise valid. This applies to every role and every protected route: an admin banning an adopter, a manager approving, declining or deactivating a staff member or vet, or an account completing a self-delete takes effect on the next request, without re-login.

> *The live status check is a DB read in addition to the JWT verification step, not a replacement for it - signature and expiry checks happen first.*

The middleware is built by one factory, buildAuthenticate({ allowPending }), which produces two exports: authenticate (allowPending false) and authenticate.allowPending (allowPending true). On routes wired with authenticate.allowPending, a Pending account whose role is in PENDING_LOGIN_ROLES is admitted; any other Pending role is still blocked. PENDING_LOGIN_ROLES = { Staff, Veterinarian, Volunteer } is defined once in server/src/services/auth/pendingRoles.js and is shared by login() and the middleware. Every other route uses plain authenticate, so a Pending staff member, vet or volunteer receives 401 everywhere else.

allowPending routes

Role	allowPending routes
Staff	GET/PUT /staff/me, GET/POST /staff/me/government-id, PATCH /staff/me/onboarding-step, PATCH /staff/me/onboarding-complete
Veterinarian	GET/PUT /vets/me, GET/POST /vets/me/government-id, PATCH /vets/me/onboarding-step, PATCH /vets/me/onboarding-complete
Volunteer	GET/PUT /volunteers/me, GET/POST /volunteers/me/government-id, PATCH /volunteers/me/onboarding-step, PATCH /volunteers/me/onboarding-complete
All three	POST /auth/logout, POST /auth/email-verification, PATCH /auth/password

The allowPending route list is deliberately minimal and has the same shape for all three roles. Approving a Pending account requires completed onboarding and a Verified government ID (see 1.6).

> *Sprint 9 added two /auth routes to the list: POST /auth/email-verification (a Pending Staff member, Veterinarian or Volunteer can verify their email while they onboard) and PATCH /auth/password (they can change their password). PATCH /auth/email-verification needs no session at all, and POST /auth/email-change uses plain authenticate, so a Pending account gets 401 there.*
>
> *Volunteers: every other volunteer route rejects a Pending volunteer with 401, including PUT /volunteers/me/availability, GET /volunteers/me/tasks, /volunteers/me/events, /volunteers/me/appointments and DELETE /volunteers/me. The /volunteers/me routes are mounted before the staff /volunteers/:id router, so that /volunteers/me is not captured by /volunteers/:id.*

1.6 Staff, Vet and Volunteer Sign-up, Onboarding and Approval

Staff, vets and volunteers register with a shelterID. The shelter must be Open (422 otherwise). A vet can register only at a shelter that has a manager: 422 "This shelter isn't onboarding veterinarians yet - it has no manager assigned".

Onboarding wizard

New staff, vets and volunteers start Pending and complete a mandatory onboarding wizard before approval. Completing onboarding (PATCH .../onboarding-complete) returns 409 naming anything that is missing.

Step	Screen
2	Personal
3	Address
4	Identity (government ID upload)
5	Review

Approval

Account	Approved by	Endpoint
Staff	Shelter Manager	PATCH /staff/me/team/:id/status
Veterinarian	Shelter Manager	PATCH /staff/me/vets/:id/status
Volunteer	Any staff member at the volunteer's shelter	PATCH /volunteers/:id/status
Manager sign-up	Admin	PATCH /staff/:id/status

Approval requires completed onboarding AND a Verified government ID; otherwise the request returns 409 naming what is missing. The rule is enforced in services/staff/staffApproval.service.js, which every approval path shares (keyed by userType Staff, Veterinarian or Volunteer). Declining is allowed at any stage and sets the account to Declined, a final status (see 1.8). A volunteer may be approved by any staff member at the volunteer's shelter, not only the Manager. Because account status is checked on every request (see 1.5), approval or deactivation takes effect on the next request without re-login.

Pre-existing approved accounts are marked as onboarded by the data migrations staff_onboarding_backfill and vet_onboarding_backfill, so the wizard applies only to new sign-ups. Existing non-Pending volunteers were likewise backfilled as onboarded.

1.7 Donor Sign-up and Skippable Onboarding

Donors register with POST /auth/register (role "donor") and no shelter. A donor account is Active from sign-up: there is no approval and no government ID, so a donor is never Pending and has no allowPending routes. Login and refresh-token responses carry onboardingComplete and onboardingStep but not accountStatus, the same as an Adopter.

Onboarding wizard

Step	Screen
2	Personal
3	Address
4	Review

Unlike the approval roles, the donor wizard is skippable. "Skip for now" lets the donor use the dashboard and donate for the current browser session, and the next login brings them back to the wizard (see 4.1, OnboardingGate). Donating never waits on onboarding.

1.8 Account Statuses (Sprint 9)

Every role table has one accountStatus column. Sprint 9 added Closed to every role's status enum and Declined to the Staff, Veterinarian, Volunteer and Admin enums (migration account_status_closed_declined). The statuses have fixed meanings, and they decide who is allowed to undo them.

Status	Meaning	Who can undo it	Login result
Closed	The owner closed their own account (DELETE /<role>/me with mode deactivate, any role).	Only the owner, through an emailed reactivation link (1.13). Nobody else can change it (409).	401 UNAUTHORIZED, error.details.reason = "account_closed" - the login page offers a reactivation link.
Declined	Turned down at approval (Staff, Veterinarian, Volunteer, Admin).	Nobody. Declined is final: the account can't be approved later.	401 UNAUTHORIZED, error.details.reason = "account_declined".
Deactivated	Done by an admin or staff member.	Only an admin or staff member (Deactivated → Active, admin endpoints).	401 UNAUTHORIZED, "This account is no longer active" (shared with Banned, see 1.2).
Banned	Applies to Adopter, Donor and Volunteer accounts.	Not reopened by the owner; restored to Active by a status change someone else makes.	401 UNAUTHORIZED, "This account is no longer active" (shared with Deactivated, see 1.2).

Status changes made by someone other than the owner follow a fixed set of transitions: Pending → Active or Declined; Active → Deactivated; Deactivated → Active (admin endpoints); and an adopter's Active, Banned and Deactivated among themselves. Anything else returns 409. Closed sits outside that set: it only changes through the owner's reactivation link. In utils/staffDates.js, Closed stamps staffDOS the same way Deactivated does; Declined stamps nothing.

> *Login and the new statuses: login checks the password first, then refuses a Closed or Declined account with a 401 that carries its own error.details.reason (account_closed, account_declined). Deactivated and Banned keep their own message. As with every login refusal this is a 401, never a 403, because the frontend's axios interceptor sends every 403 to /forbidden (see 1.2).*
>
> *Seed data covers every status for every role (the *.closed@ and *.declined@ accounts), plus an unverified adopter and donor. Seeded addresses are fake @petpals.com ones, so never act on them with a real email provider key set.*

1.9 Emailed-Link Tokens (Sprint 9)

Four emailed links share one mechanism: verify an email address, reset a password, reactivate a Closed account and confirm an email change. Their tokens live in the AccountToken table (migration account_tokens) and are managed by services/auth/accountTokens.service.js, which exports issueToken, consumeToken and hashToken.

How a token works

Property	Behaviour
Raw token	32 random bytes encoded as base64url. It exists only inside the emailed link - never in an API response and never in the database.
What is stored	Only its SHA-256 hex (tokenHash, VARCHAR(64), UNIQUE), so a leaked table can't be turned back into working links. The row also holds userID, purpose, expiresAt, usedAt and, for EmailChange only, newEmail. It is deleted with its user (ON DELETE CASCADE from Users).
Issuing	issueToken(userID, purpose) deletes the user's earlier unused tokens of that purpose, plus any of their expired ones, then creates the new token. Only the newest link works.
Resend cooldown	60 seconds per user and purpose (RESEND_COOLDOWN_MS). Inside it, issueToken throws TOO_MANY_REQUESTS (429) with error.details.retryAfterSeconds. The check runs before anything is deleted, so the earlier link keeps working. Message: "An email was just sent — please wait N seconds before asking for another".
Redeeming	consumeToken(rawToken, purpose) trims and hashes the raw token, then checks in order:

	unknown or for a different purpose → token_invalid; already used → token_used; expired → token_expired. It marks usedAt with a conditional update (usedAt must still be null), so two racing requests can't both succeed: the second gets token_used. Returns { userID, newEmail }.
Same transaction	Both functions accept the caller's Prisma $transaction client (db). Every endpoint consumes its token inside the same transaction as the change the link authorises, so a failed change leaves the link usable.

Link lifetimes

Purpose	Lifetime	Used by
EmailVerification	24 hours	Email verification (1.10)
PasswordReset	1 hour	Forgot password → reset (1.11). Shortest life: a reset link sets a new password without the old one.
Reactivation	24 hours	Reopening a Closed account (1.13)
EmailChange	24 hours	Confirming a new login email (1.14)

The lifetimes and the cooldown are defined once in config/email.js (TOKEN_TTL_MS, RESEND_COOLDOWN_MS). The cooldown stops someone hammering "Resend" into a stranger's inbox.

Link failure reasons

error.details.reason	When	Message
token_invalid	Token missing or not a string, unknown, or issued for a different purpose	This link is invalid
token_used	The link was already redeemed	This link has already been used
token_expired	The link is past its lifetime	This link has expired — please request a new one

All three are BAD_REQUEST (400). The reason lets the client tell "request a new link" from "already done". How the emails carrying these links are built and sent (templates, queueEmail after the DB commit, provider, console fallback in development) is covered in 16 - Email & Account Security and is not repeated here.

1.10 POST and PATCH /api/v1/auth/email-verification (Sprint 9)

Registration already sends the first verification link; POST is the resend and PATCH redeems a link. Opening a link proves the address, so it sets USERS.emailVerified to true. Everyone can log in unverified: only applying to adopt and donating need a verified email (see 1.15).

Request summary - POST (send a new link)

Field	Value
Method	POST
Auth required	Yes - Bearer access token, via authenticate.allowPending (a Pending Staff member, Veterinarian or Volunteer can verify while they onboard)
Body fields	None
Success	200 - data is null. Each new link replaces the previous unused one and expires in 24 hours.

Request summary - PATCH (redeem the link)

Field	Value
Method	PATCH
Auth required	No - no session needed; the link may be opened on another device
Body fields	token
Success	200 - { emailVerified: true }. Each link works once.

Data flow

File / Layer	What happens here
auth.routes.js	POST /email-verification → authenticate.allowPending → authController.requestEmailVerification. PATCH /email-verification → authController.verifyEmail, no middleware.
accountTokens.service.js	POST: issueToken(userID, "EmailVerification") - 60-second cooldown (429), earlier unused links deleted, new link valid for 24 hours. PATCH: consumeToken(token, "EmailVerification") inside the same $transaction as the emailVerified update.
Email	The link is emailed after the write commits (queueEmail); see 16 - Email & Account Security.
USERS table	Column written on PATCH: emailVerified = true.

Error cases

Scenario	Error Code	Response details
POST: not signed in	UNAUTHORIZED (401)	Missing or invalid access token
POST: email is already verified	CONFLICT (409)	error.details.reason = "already_verified"
POST: another link asked for within 60 seconds	TOO_MANY_REQUESTS (429)	error.details.retryAfterSeconds says how long to wait
PATCH: Link invalid, expired or already used	BAD_REQUEST (400)	error.details.reason = "token_invalid", "token_expired" or "token_used" (see 1.9)

1.11 POST and PATCH /api/v1/auth/password-reset (Sprint 9)

Forgot password is two steps: POST asks for a link, PATCH sets the new password with the token from it. Neither needs a session.

Request summary - POST (forgot password)

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	email
Success	200 - data is null, with the same message whether or not the email has an account (see 1.16). A link is only sent to an existing account that isn't Banned or Declined, at most once a minute (a quicker repeat sends nothing). The link expires in 1 hour and replaces any earlier unused one.

Request summary - PATCH (set a new password)

Field	Value
Method	PATCH
Auth required	No - the token is the proof
Body fields	token, password (the five password rules, see 1.16)
Success	200 - { role }, so the page can link to the right login. Signs out every session (stored refresh token cleared, cookie cleared), marks the email verified and emails a "password changed" notice.

Flow: forgot → reset

```
ForgotPassword page → POST /auth/password-reset { email } → same 200 for any
email → emailed link (existing, non-Banned, non-Declined account only)
```

```
→ ResetPassword page (?token=) → PATCH /auth/password-reset { token,
password } → new password, every session signed out, email verified
```

File / Layer	What happens here
ForgotPassword (/forgot-password)	The person enters their email and the page calls POST /auth/password-reset. ?portal=staff keeps the worker-portal look.
auth.routes.js	Public routes, no middleware: POST → authController.requestPasswordReset, PATCH → authController.resetPassword.
Request step	Returns VALIDATION_ERROR (422) if email is missing. Otherwise always answers 200 with the same message. Only an existing account that isn't Banned or Declined gets a link, and a repeat inside a minute sends nothing. issueToken(userID, "PasswordReset") deletes earlier unused reset links; the new one lasts 1 hour. The email is queued after the commit (16 - Email & Account Security).
ResetPassword (/reset-password)	The link opens this page with ?token=. Unlike the other link pages it does not redeem on mount, because it needs the new password first; it then calls PATCH /auth/password-reset.
Reset step	consumeToken(token, "PasswordReset") runs in the same $transaction as the password change, so a failed change leaves the link usable. A weak password returns VALIDATION_ERROR (422) with error.details.unmet. On success: every session is signed out (stored refresh token cleared, cookie cleared), emailVerified is set to true, a "password changed" notice is emailed, and { role } is returned.

Error cases

Scenario	Error Code	Response details
POST: email missing	VALIDATION_ERROR (422)	email missing
PATCH: Link invalid, expired or already used	BAD_REQUEST (400)	error.details.reason = "token_invalid", "token_expired" or "token_used" (see 1.9)
PATCH: password too weak	VALIDATION_ERROR (422)	error.details.unmet lists the missing rules

1.12 PATCH /api/v1/auth/password (Sprint 9)

Changes the password of a signed-in user. Unlike a reset it needs the current password, and it keeps the caller signed in.

Request summary

Field	Value
Method	PATCH
Auth required	Yes - Bearer access token, via authenticate.allowPending (a Pending Staff member, Veterinarian or Volunteer can change it while they onboard)
Body fields	currentPassword, newPassword (same rules as a reset; must differ from the current one)
Success	200 - data is null. This session stays signed in (a new refresh token cookie is set); any other session is signed out. Emails a "password changed" notice.

Error cases

Scenario	Error Code	Response details
Not signed in	UNAUTHORIZED (401)	Missing or invalid access token
Current password is wrong	UNAUTHORIZED (401)	error.details.reason = "wrong_password"
New password too weak	VALIDATION_ERROR (422)	error.details.unmet lists the missing rules
New password is the same as the current one	VALIDATION_ERROR (422)	-

1.13 POST and PATCH /api/v1/auth/reactivation (Sprint 9)

Lets the owner of a Closed account reopen it. POST asks for a link and PATCH redeems it. Neither needs a session.

Request summary - POST (ask for a link)

Field	Value
Method	POST
Auth required	No - public endpoint
Body fields	email
Success	200 - data is null, always with the same message. A link is only sent when the email belongs to a Closed account (closed by its owner) - never to one an admin or staff member deactivated, one declined at sign-up or a banned one - at most once a minute. The link expires in 24 hours.

Request summary - PATCH (reopen the account)

Field	Value
Method	PATCH
Auth required	No - the token is the proof
Body fields	token
Success	200 - { role, accountStatus } where accountStatus is Active or Pending (see the table below). Also marks the email verified.

Outcome by role

Role	Status after	What happens next
Adopter	Active	Goes straight back to Active
Donor	Active	Goes straight back to Active
Staff	Pending	Re-approved the usual way (see 1.6)
Veterinarian	Pending	Re-approved the usual way (see 1.6)
Volunteer	Pending	Re-approved the usual way (see 1.6)
Admin	Pending	Re-approved the usual way; a Pending Admin can't log in (see 1.2)

Flow: close → reactivate

```
DELETE /<role>/me { mode: "deactivate" } → Closed, refresh token cleared →
login refused: 401 account_closed → ReactivationOffer
→ POST /auth/reactivation { email } → emailed link (24 hours) → /reactivate (?
token=) → PATCH /auth/reactivation { token } → Active, or Pending for workers
```

File / Layer	What happens here
Close	The owner calls DELETE /<role>/me with mode deactivate (see 3.6). The account becomes Closed, the refresh token is cleared and a "closed" notice is emailed after the commit (16 - Email & Account Security).
Login refused	POST /auth/login with the correct password returns 401 with error.details.reason = "account_closed".
ReactivationOffer	Login and WorkerLogin show ReactivationOffer when they see account_closed, so the person can ask for a link.
Request step	POST /auth/reactivation returns the same 200 for any email (VALIDATION_ERROR 422 if email is missing). issueToken(userID, "Reactivation") only runs for a Closed account; a repeat inside a minute sends nothing.
/reactivate	The link opens this page with ?token=; it redeems the token once on mount (a ref guard stops React StrictMode redeeming it twice).

Redeem step	PATCH /auth/reactivation runs consumeToken(token, "Reactivation") in the same $transaction as the status change. Adopter and Donor → Active; Staff, Veterinarian, Volunteer and Admin → Pending. The email is marked verified and { role, accountStatus } is returned.

Error cases

Scenario	Error Code	Response details
POST: email missing	VALIDATION_ERROR (422)	email missing
PATCH: Link invalid, expired or already used	BAD_REQUEST (400)	error.details.reason = "token_invalid", "token_expired" or "token_used" (see 1.9)
PATCH: the account is no longer Closed	CONFLICT (409)	error.details.reason = "not_closed"; the link stays unused

1.14 POST and PATCH /api/v1/auth/email-change (Sprint 9)

Changes the login email in two steps. POST asks for the change and nothing changes until the link sent to the new address is opened; PATCH confirms it.

Request summary - POST (ask for the change)

Field	Value
Method	POST
Auth required	Yes - Bearer access token, via plain authenticate (a Pending account gets 401 here)
Body fields	newEmail (max 45 characters), currentPassword
Success	200 - data is null. Sends a confirmation link to the new address and a heads-up to the current one. The link expires in 24 hours and a newer request replaces it.

Request summary - PATCH (confirm the change)

Field	Value
Method	PATCH
Auth required	No - no session needed
Body fields	token
Success	200 - { userEmail }. Re-checks the address is still free, sets it as the login email (verified) and tells the old address it changed.

Data flow

File / Layer	What happens here
auth.routes.js	POST /email-change → authenticate → authController.requestEmailChange. PATCH /email-change → authController.confirmEmailChange, no middleware.
accountTokens.service.js	POST: issueToken(userID, "EmailChange", { newEmail }) stores the new address on the AccountToken row (24-hour lifetime, 60-second cooldown). PATCH: consumeToken(token, "EmailChange") returns { userID, newEmail } inside the same $transaction as the email update.
Email	The confirmation link goes to the new address and the heads-up to the current one; after the change the old address is told it changed. All are queued after the commit (16 - Email & Account Security).

Error cases

Scenario	Error Code	Response details
POST: not signed in	UNAUTHORIZED (401)	Missing or invalid access token

POST: current password is wrong	UNAUTHORIZED (401)	error.details.reason = "wrong_password"
POST: another account already uses the address	CONFLICT (409)	error.details.reason = "email_taken"
POST: newEmail missing, invalid, longer than 45 characters or the same as the current one	VALIDATION_ERROR (422)	-
POST: asked again within 60 seconds	TOO_MANY_REQUESTS (429)	error.details.retryAfterSeconds
PATCH: Link invalid, expired or already used	BAD_REQUEST (400)	error.details.reason = "token_invalid", "token_expired" or "token_used" (see 1.9)
PATCH: someone else took the address in the meantime	CONFLICT (409)	error.details.reason = "email_taken"

1.15 Soft Verification Gate (Sprint 9)

Everyone can log in with an unverified email and use the dashboard, where a banner offers to send the link. Only applying to adopt and donating need a verified email.

Action	Endpoint	Verified email needed?
Log in, browse, use the dashboard	POST /auth/login and every other route	No
Apply to adopt a pet	POST /adoption-applications	Yes - otherwise 409 CONFLICT, error.details.reason = "email_unverified"
Donate	POST /donations/checkout	Yes - otherwise 409 CONFLICT, error.details.reason = "email_unverified"

The check runs before any pet or shelter lookup and before any Stripe call. Registration sends the first verification link, and accounts that existed before Sprint 9 were backfilled as verified.

> *Why 409 and not 403: the frontend's axios interceptor redirects every 403 to /forbidden, so a 403 would send the person away from the page. A 409 CONFLICT with error.details.reason = "email_unverified" does not.*

On the client, AuthUser.emailVerified drives the EmailVerificationBanner (top of main in the shared DashboardLayout) and EmailUnverifiedNotice, which replaces the submit button in AdoptApply and DonateForm. A 409 email_unverified also flips emailVerified in the session. Every Verify or Resend button goes through useResendVerification: it shows the toasts, treats a 409 already_verified as "the session catches up", and shows the wait on a 429.

1.16 Security Decisions (Sprint 9)

Decision	What it means
No account enumeration	POST /auth/password-reset and POST /auth/reactivation always return 200 with the same message, whether or not the email has an account, so they can't be used to find out who is registered. A link is only sent to an eligible account (reset: exists and isn't Banned or Declined; reactivation: Closed only).
Links are single-use and hashed	Only the SHA-256 of a token is stored, each link works once, and only the newest link of a purpose works (see 1.9).
A reset signs out every session	PATCH /auth/password-reset clears the stored refresh token and the cookie, and emails a "password changed" notice.
A change keeps this session	PATCH /auth/password sets a new refresh token cookie for the current session and signs out any other session.
A link proves the address	Opening a reset, reactivation or email-change link also marks the email verified.
Current password for sensitive changes	Changing the password and asking for an email change both need the current password (401 wrong_password).
One set of password rules	Register, reset and change enforce the same five rules on the server: 8+ characters with an uppercase letter, a lowercase letter, a number and a special character

	(utils/password.js, mirrored in the client's password.ts). A weak password is a 422 whose error.details.unmet lists the missing rules.
Heads-up to the old address	An email-change request tells the current address, and a confirmed change tells the old address that it changed.
401 and 409, not 403	The axios interceptor sends every 403 to /forbidden, so login refusals are 401 and the verification gate is 409.

2. Backend Supporting Files

Shared across all auth endpoints

File	Purpose
config/prisma.js	Singleton Prisma client using PrismaPg adapter. Single gateway to the DB - imported by all service files.
utils/response.js	successResponse() and errorResponse() - standardise all API response shapes.
utils/errors.js	ERROR_CODES map - defines valid error codes and their HTTP status codes (400, 401, 403, 404, 409, 422, 429, 500). 429 TOO_MANY_REQUESTS is used when another emailed link is requested within a minute (see 1.9).
middleware/errorHandler.js	Global error handler registered last in app.js. Receives errors via next(err), maps err.code to HTTP status, formats using errorResponse().
middleware/authenticate.js	JWT verification and live account-status middleware, built by the buildAuthenticate({ allowPending }) factory. Extracts Bearer token, verifies signature, attaches { userID, role } to req.user - then calls authService.getAccountStatus(userID, role) and rejects with 401 if the account is Deactivated, Banned, Pending, or DELETED (see 1.5). Exports authenticate (allowPending false) and authenticate.allowPending (allowPending true: also admits Pending accounts whose role is in PENDING_LOGIN_ROLES). Used on logout, POST /auth/email-verification and PATCH /auth/password (allowPending variant), and on POST /auth/email-change (plain variant). Not used on refresh-token.
middleware/authorizeRoles.js	RBAC middleware. Takes ...allowedRoles, checks req.user.role against the list, returns 403 FORBIDDEN if not permitted. Applied to every role-specific endpoint.
services/auth/pendingRoles.js	Defines PENDING_LOGIN_ROLES = { Staff, Veterinarian, Volunteer } once - the roles whose Pending accounts can log in and reach their allowPending routes. Shared by login() in auth.service.js and the authenticate middleware.
services/staff/ staffApproval.service.js	Single approval rule shared by every staff, vet and volunteer approval path (keyed by userType Staff, Veterinarian or Volunteer): onboarding complete AND government ID Verified, otherwise 409 naming what is missing. Declining is always allowed.
app.js	Express app setup - middleware, routes, error handler. No app.listen() call. Imported by tests directly.
index.js	Thin entry point - requires app.js and calls app.listen(). Split from app.js to allow Supertest to import the app without binding a port.
services/auth/ accountTokens.service.js	issueToken, consumeToken and hashToken for the emailed links: random token, only its SHA-256 stored, single use, newest link wins, 60-second resend cooldown (see 1.9).
config/email.js	Email knobs in one place: link lifetimes per purpose (TOKEN_TTL_MS), the resend cooldown (RESEND_COOLDOWN_MS), the provider timeout and the default sender name. The sending side is covered in 16 - Email & Account Security.
utils/password.js	The five password rules, enforced on register, reset and change. Mirrored on the client by logic/utils/password.ts (see 1.16).
services/email/	Email sending (templates, queueEmail, provider, console fallback). Not covered here - see 16 - Email & Account Security.

3. Backend Supporting Files — client/src | Sprint 1

The frontend auth system is built around four files that each have a single responsibility. They are intentionally decoupled - the API layer knows nothing about state, the state layer knows nothing about HTTP, and the UI layer composes both.

3.1 File responsibilities

File	Responsibility
api/authApi.ts	API functions for auth endpoints - register(), login(), refreshToken(), logout(). Each makes an axios call and returns typed data. Knows nothing about state.
api/axiosInstance.ts	Pre-configured axios client. Sets baseURL from VITE_API_URL env variable, withCredentials: true (sends httpOnly cookie automatically). Request interceptor attaches Bearer token from Zustand. Response interceptor handles 401 (clear state + redirect to the login page for the role: /staff-portal/login for Admin/Staff/Veterinarian, /login otherwise) and 403 (redirect to /forbidden), skipping auth endpoints to avoid redirect loops.
store/useAuthStore.ts	Zustand global store. Holds { user, token, role } in memory. login() sets all fields, logout() clears them. State is lost on page refresh - restored via refreshToken() on app load.
App.tsx	Root component. On mount, calls refreshToken() to restore session from the httpOnly cookie. On success, populates Zustand. On failure, does nothing - user will see public pages or be redirected to /login by ProtectedRoute. Uses authReady state to prevent flash of unauthenticated content.

3.2 Session restore on page load

Because Zustand state is memory-only, it is lost on every page refresh. The session is restored using the httpOnly cookie which persists across refreshes.

Flow

File / Layer	What happens here
App.tsx	useEffect fires on mount, calls refreshToken(). authReady state starts false - renders a rose background placeholder to prevent flash. refreshToken() called - axiosInstance sends the request with credentials: true. Browser automatically attaches the httpOnly refreshToken cookie
axiosInstance.ts	Sends POST /auth/refresh-token with withCredentials: true. isAuthCall guard on response interceptor prevents redirect loop if this call returns 401
Backend /auth/refresh-token	Verifies cookie, rotates tokens, returns new access token + name. New refreshToken set as httpOnly cookie in response. Returns { token, user: { userID, role, name, avatarSeed, emailVerified, onboardingComplete, onboardingStep, accountStatus } }
App.tsx .then()	On success, populates Zustand and sets authReady = true. storeLogin(user, token, user.role) - stores session in memory. authReady = true - full app renders, Navbar shows user name and dropdown
App.tsx .catch()	On failure (no cookie, expired), silently ignored. authReady = true - app renders normally. User sees public pages; ProtectedRoute redirects to /login if they try to access a dashboard

3.3 Login flow (Login.tsx)

File / Layer	What happens here
Login.tsx	Validates fields client-side, calls authApi.login(), updates Zustand, navigates. emailRegex validation + required field check before hitting API. noValidate on form - browser validation suppressed, custom error box used. On success: if a ?redirect= param is present, go there (unless it points into another role's dashboard); otherwise go to the role's dashboard via resolveDestination (Veterinarian → /vet, Volunteer → /volunteer, Donor → /donor). replace: true prevents back button returning to login page. On failure: displays server error message inline. If user already authenticated (token in Zustand): <Navigate> redirect away from login immediately. Login and WorkerLogin also show a "Forgot password?" link (/forgot-password; ?portal=staff keeps the worker look) and, when the server answers with

	error.details.reason "account_closed", a ReactivationOffer so the owner can ask for a reactivation link (see 1.13)
authApi.ts (login)	POSTs { email, password } to /auth/login. Returns { token, user: { userID, userEmail, role, name, avatarSeed, emailVerified, onboardingComplete, onboardingStep, accountStatus } }. axiosInstance automatically receives and stores the httpOnly refreshToken cookie

3.4 Logout flow (Navbar.tsx)

File / Layer	What happens here
Navbar.tsx (handleLogout)	Calls the API, then leaves and clears the session in ONE render via useEndSession (flushSync around navigate + storeLogout) — clearing the session first would let ProtectedRoute capture the current page as a stale /login?redirect. Destination by role comes from logoutDestinationFor: Admin, Staff and Veterinarian → /staff-portal/login; everyone else (Adopter, Volunteer, Donor) → /.
authApi.ts (logout)	POSTs to /auth/logout with Bearer token from Zustand. axiosInstance request interceptor attaches Authorization: Bearer <token> automatically

3.5 Worker session flow and destinations

Session restore (3.2) rebuilds the session from POST /auth/refresh-token on every app load. The response carries the onboarding fields and accountStatus that the OnboardingGate (4.1) reads to route a Pending staff member, vet or volunteer: unfinished onboarding leads to the onboarding wizard (/staff/onboarding/step/:step, /vet/onboarding/step/:step or /volunteer/onboarding); finished onboarding with approval pending leads to the shared AwaitingApproval page (/staff/pending, /vet/pending or /volunteer/pending); an Active account reaches its dashboard.

Workers (Admin, Staff, Veterinarian) land on the worker login after logging out, after a 401, and after closing their own account. Adopters, Volunteers and Donors go home (/) on log out and to /login on a 401. Volunteers and Donors are not workers: they sign in on the public /login page, closing their own account also ends on /, and after login they land on /volunteer and /donor respectively.

Event	Workers (Admin, Staff, Veterinarian)	Adopters, Volunteers and Donors
Log out	Worker login (/staff-portal/login)	Home (/)
After a 401	Worker login (/staff-portal/login)	/login
After closing their own account	Worker login (/staff-portal/login)	Home (/) (Volunteers, Donors)

The destinations come from three helpers in client/src/logic/route/resolveDestination.ts: logoutDestinationFor, loginPathFor and loginPathForPage. A signed-out visitor who opens /staff, /admin or /vet is sent to the worker login with ?redirect= preserved. A signed-out visitor who opens /volunteer or /donor is sent to /login with ?redirect= preserved.

Ending a session from the UI always goes through useEndSession, which navigates and clears the session inside one flushSync so ProtectedRoute never captures a stale redirect.

3.6 Closing your own account (session side)

DELETE /vets/me, DELETE /staff/me, DELETE /volunteers/me and DELETE /donors/me take { mode: "deactivate" | "delete" } and use plain authenticate, so a Pending account cannot call them (401). deactivate clears the refresh token and sets accountStatus to Closed (see 1.8), which its owner can undo through an emailed reactivation link (see 1.13); delete removes the account's Users row. DELETE /volunteers/me returns 409 with error.details.blockers (["appointments", "tasks"], whichever apply) while the volunteer has upcoming assisted appointments or open tasks; DELETE /donors/me is never blocked. The client then ends the session through useEndSession: Staff and Veterinarians land on the worker login, Volunteers and Donors land on /.

4. Role-Based Access Control — Frontend guards + backend middleware

PetPals enforces RBAC at two layers - the frontend prevents unauthenticated or wrong-role navigation, and the backend enforces the same rules on every API request. Both must be in place: frontend guards are UX, backend enforcement is security.

4.1 Frontend route guards

ProtectedRoute

Wraps any route that requires authentication. Reads token from Zustand - if absent, redirects to /login?redirect=<path> — or to /staff-portal/login?redirect=<path> for worker dashboards (/staff, /admin, /vet); /volunteer and /donor are not worker dashboards and use /login. If present, renders children.

```
Usage: <ProtectedRoute><AdopterDashboard /></ProtectedRoute>
```

RoleRoute

Wraps routes that require a specific role. Accepts an allowedRoles prop. Reads role from Zustand - if role is not in the allowed list, redirects to /forbidden. Used inside ProtectedRoute.

```
Usage: <ProtectedRoute><RoleRoute allowedRoles={['Adopter']}><AdopterDashboard
/></RoleRoute></ProtectedRoute>
```

OnboardingGate

Wraps every route and holds a signed-in user in their onboarding until it is done. There are two kinds of role. Approval roles (Staff, Veterinarian, Volunteer) are held in their wizard until onboarded, then on the shared AwaitingApproval page until approved; an Active account reaches its dashboard. Skippable roles (Adopter, Donor) are sent to their wizard while onboarding is unfinished, but may choose "Skip for now".

"Skip for now" sets a sessionStorage flag that lifts the gate for the current browser session only. For an Adopter, /adopt/apply stays gated even after skipping; a Donor has no gated route, so a skipped Donor can use everything, including donating. The flag is cleared on the next real login, which brings the user back to the wizard.

The five emailed-link pages (/verify-email, /forgot-password, /reset-password, /reactivate and /confirm-email-change) are listed in OnboardingGate's LINK_PAGES, so a user who is mid-onboarding or Pending isn't redirected away before the token in the link is used. Any new link page must be added there.

Role	Type	Wizard	Awaiting approval	Skip for now
Staff	Approval	/staff/onboarding	/staff/pending	No
Veterinarian	Approval	/vet/onboarding	/vet/pending	No
Volunteer	Approval	/volunteer/onboarding	/volunteer/pending	No
Adopter	Skippable	/onboarding	-	Yes (/adopt/apply stays gated)
Donor	Skippable	/donor/onboarding	-	Yes (no gated route)

Route configuration in App.tsx

Route	ProtectedRoute	RoleRoute allowedRoles
/adopter	Yes	Adopter
/staff	Yes	Staff
/vet	Yes	Veterinarian
/volunteer	Yes	Volunteer
/donor	Yes	Donor
/admin	Yes	Admin
/login	No (redirects if already logged in)	-
/register	No (redirects if already logged in)	-

/verify-email	No	-
/forgot-password	No	-
/reset-password	No	-
/reactivate	No	-
/confirm-email-change	No	-
/forbidden	No	-

4.2 Backend RBAC middleware

authorizeRoles.js

Factory middleware that accepts a list of allowed roles and returns an Express middleware function. Called after authenticate so req.user.role is already set.

```
Usage: router.post('/pets', authenticate, authorizeRoles('Staff', 'Admin'),
createPet)
```

Applied to every protected resource endpoint.

ROLES constant

Key	Value (DB enum)
ROLES.ADMIN	Admin
ROLES.STAFF	Staff
ROLES.ADOPTER	Adopter
ROLES.VETERINARIAN	Veterinarian
ROLES.VOLUNTEER	Volunteer
ROLES.DONOR	Donor

4.3 Forbidden page

A simple public page at /forbidden rendered when RoleRoute or the axiosInstance 403 handler redirects an authenticated user who lacks the required role. Uses the same background image and card style as the login page.

Triggered by: RoleRoute (wrong role on frontend route) and axiosInstance response interceptor (403 from any API call).

5. Auth System Tests — Jest + Supertest | Sprint 1

Test file	Type	Cases covered
tests/unit/auth/authenticate.test.js	Unit	No token → 401, Invalid JWT → 401, Expired JWT → 401, Valid JWT → sets req.user and calls next(), allowPending admits only Pending Staff, Veterinarian and Volunteer accounts (any other Pending role stays blocked)
tests/unit/authorizeRoles.test.js	Unit	Role in allowedRoles → next(), Role NOT in allowedRoles → 403, Multiple allowed roles each pass
tests/unit/auth/login.test.js	Unit	Pending Staff/Vet/Volunteer login and refresh → 200 with accountStatus and onboarding progress; Deactivated → 401; Pending Admin → 401 with no session cookie
tests/unit/vet/ vets.selfService.test.js	Unit	Pending vets reach only their onboarding routes (GET/PUT /vets/me, government ID, onboarding-step, onboarding-complete); every other route → 401
tests/unit/staff/ staff.selfService.test.js	Unit	Pending staff reach only their onboarding routes (GET/PUT /staff/me, government ID, onboarding-step, onboarding-

		complete); every other route → 401
tests/integration/ auth.register.test.js	Integration	Valid payload → 201 + DB rows created, Missing fields → 422, Duplicate email → 409, Password hashed in DB
tests/integration/auth.login.test.js	Integration	Valid credentials → 200 + JWT, Wrong password → 401, Unregistered email → 404, JWT contains correct userID and role
tests/integration/ auth.logout.test.js	Integration	Valid token → 200 + DB refreshToken null, Stale refresh token after logout → 401, No token → 401
tests/integration/ auth.refreshToken.test.js	Integration	Valid cookie → 200 + new JWT
tests/integration/staff/ staffOnboarding.journey.test.js	Integration	Sign up → Pending login → onboard → ID verified → manager approves → full access
tests/integration/vet/ sprint6Journeys.test.js	Integration	Register → Pending login → onboarding → government ID → manager verifies ID → manager approves → full access
tests/integration/auth/ sprint9Journeys.test.js	Integration	Email and account-security journeys end to end: services/email is mocked so every email is captured and its link followed. The jest setup file noEmailProvider.js blanks the EMAIL_* settings so no test can reach the real provider

All integration tests seed data via the real register/login endpoints and clean up via Prisma in afterAll. Tests run sequentially (--runInBand) to avoid FK conflicts on the shared Supabase dev DB.